SE 413/513: Software Testing

Assignment 2: Coverage, Data-Flow Testing, and Test Doubles

Total Points: 7.5 **Submission:** GitHub repository and branch

Platform: Python with `pytest` and `pytest-cov` is the default and assumed throughout this document. A different programming language/testing framework may be used only with prior instructor approval, and must still demonstrate the equivalent concepts (statement/branch coverage reporting, data-flow/DU-pair analysis, test doubles).

Coverage: Unlike Assignment 1, full statement and branch coverage **is** required for the two functions this assignment targets — but there is no fixed numeric threshold to hit. Run the coverage report, read the `Missing` column, and keep adding tests until it shows no gaps for those two functions.

1. The Practical Problem

This assignment extends the same **Course Registration System** from Assignment 1. `src/registration.py` still provides `can_register()` and `calculate_registration_fee()` unchanged, and the starter code adds **two new, fully implemented functions** that build on them. **You will not modify any of this business logic.** Your work is to design and implement a thorough `pytest` suite that achieves full statement and branch coverage of the two new functions, demonstrates the underlying data-flow (def/use) coverage for one of their variables, and uses test doubles to isolate a collaborator.

1.1 Batch Registration Summary

The registrar's office wants to process a batch of registration requests at once and get back how many were approved and the total fees collected. Each request in the batch that `can_register()` would approve contributes its resulting registration fee to a running total; rejected requests contribute nothing.

```
summarize_batch_registrations(requests) # -> (approved_count, total_fees)
```

`requests` is a list of `(current_credits, course_credits, prerequisite_met)` tuples, one per student request.

1.2 Enrollment with Notification

The registration system also needs to notify a student of the outcome of a single enrollment attempt. If `can_register()` allows it, the student is enrolled and told the fee they owe; otherwise they are told the request was rejected. The notification itself is delegated to a `notifier` collaborator so that `enroll_student()` does not need to know *how* a student is notified (email, push notification, etc.) — only *that* they are.

```
enroll_student(current_credits, course_credits, prerequisite_met, notifier) # -> bool
# On success: notifier.send_confirmation(fee) is called, returns True
# On failure: notifier.send_rejection() is called, returns False
```

2. GitHub Project and Submission Requirements [0.25 points]

Create a repository, add a branch named exactly `assignment-2`, and give the instructor (GitHub ID `mdkamrulhasan`) access. All work must be committed and pushed to `assignment-2`.

Submit your GitHub branch link, including a reference to your final commit (commit hash or message). Do not update the branch after your final submission — the **latest push timestamp on `assignment-2` is treated as your submission date**; use a different branch if you keep working.

3. Set Up the Starter Project [0.25 points]

```
assignment-2/
|-- src/registration.py (provided, fully implemented)
|-- tests/test_registration.py
|-- tests/conftest.py
|-- pyproject.toml
|-- README.md
```

Clone the provided starter project and confirm that `pytest` runs cleanly from the project root. `pyproject.toml` is preconfigured for branch coverage measurement scoped to `src/` (see `[tool.coverage.run]`) and still registers the four markers from Assignment 1 (`unit`, `positive`, `negative`, `boundary`) if you find them useful here.

4. Statement and Branch Coverage [2.25 points]

Measure coverage of `summarize_batch_registrations()` and `enroll_student()` with:

```
pytest --cov --cov-branch --cov-report=term-missing
```

The report's `Missing` column lists uncovered line numbers and `a->b` arcs (uncovered branch outcomes). Because the report measures the whole file, it will also mention `can_register`/`calculate_registration_fee` — those are **not** this assignment's target; only close gaps inside the two new functions.

(a) **[1.25]** Write tests until the report shows **0% missing statements** for `summarize_batch_registrations` and `enroll_student`. Include the final `term-missing` output (statement-only, i.e. drop `-cov-branch`) in your submission.

(b) **[1]** Write tests until the report shows **no missing branches** for both functions with `-cov-branch` enabled. Include the final branch-coverage output, and for each decision in the two functions (the `for` loop and each `if`), state in one sentence which test(s) exercise each of its outcomes.

5. Data-Flow Testing: `total_fees` [2.25 points]

Focus this section on the `total_fees` variable inside `summarize_batch_registrations()`. Recall from lecture: a *definition* (def) is a statement that assigns a variable; a *use* is a statement that reads it (a *c-use*, in a computation, or a *p-use*, in a predicate); a *DU pair* is a definition together with a use it reaches along a def-clear path.

(a) **[0.75]** Identify every definition and every use of `total_fees` in `summarize_batch_registrations()` (there are two definitions: the initialization before the

loop, and the accumulation inside it; and two uses: the accumulation itself and the `return` statement).

(b) **[0.75]** Build a DU-pair table listing each (definition, use) pair you identified, including the **loop-carried pair** where the accumulation definition on one iteration is read by the accumulation use on a *later* iteration. For each pair, name the test that exercises it.

(c) **[0.75]** Write the test for the loop-carried pair specifically: a batch with **at least two approved requests**, asserting the final `total_fees` is the correct sum. In 1-2 sentences, explain why a batch with zero or one approved request cannot exercise this pair (All-Uses coverage requires it; All-Defs alone would not).

6. Test Doubles for the Notifier Collaborator [2 points]

`enroll_student()` depends on a `notifier` collaborator it does not construct itself — a seam you can exploit in your tests instead of wiring up a real email/notification service.

(a) **[0.5]** In `tests/conftest.py`, create a fixture that returns a plain `unittest.mock.Mock` standing in for `notifier`.

(b) **[0.75]** Write a positive test for the allowed case using **both** state verification (assert the return value is `True`) and behavior verification (assert `send_confirmation` was called once with the correct fee, and `send_rejection` was never called).

(c) **[0.75]** Write a negative test for the rejected case using the same two kinds of verification (return value `False`; `send_rejection` called once with no arguments; `send_confirmation` never called).

This assignment only requires a plain `Mock` used as a stub-and-spy; you do not need `MagicMock`, `spec`/`autospec`, or `patch()`.

7. Organize and Document the Test Suite [0.5 points]

(a) **[0.25]** Apply at least one marker (from Assignment 1's set, or one of your own) to each new test, and document in `README.md` the commands for running the full suite, the coverage commands from Section 4, and at least one marker-filtered subset.

(b) **[0.25]** Demonstrate the full suite passes from the project root.

8. Deliverables Checklist *[No additional points]*

Before your final push, confirm the repository has: the provided `src/registration.py` unmodified; `tests/test_registration.py` and `tests/conftest.py`; `pyproject.toml` and `README.md`; your final statement- and branch-coverage reports for `summarize_batch_registrations` and `enroll_student` showing no gaps; your `total_fees` def/use identification and DU-pair table; a `Mock`-based notifier fixture with both a positive and a negative test using state *and* behavior verification; and evidence the full suite passes.

Recall: submit your GitHub branch link with a reference to your final commit, and do not update the branch afterward.

Guiding principle: Coverage tells you *whether* your tests exercised the code; data-flow analysis tells you *which variable interactions* are worth targeting beyond coverage alone; test doubles let you test a unit's own logic *in isolation* from the collaborators it talks to.